Distributed Peer-to-Peer Disaster Relief Communication Mesh

TE Sem V Mini Project I — Department of Computer Engineering, SPIT Academic Year 2026–27, Odd Semester · Group 20 Team: Twissha Shah · Miti Shah · Tanvi Sheth

This is the master reference document. Everything from scope to demo script lives here. When a decision is disputed, this file wins. Update it in place rather than starting new documents.

0. Start here

If you are opening a new AI chat: upload this file as your first message and say who you are, which track you are on, and what you are working on. For example — *"I'm Tanvi, Track A. I'm writing packet.py for Phase 1. Read the doc and help me."* That is enough for full context.

0.1 The project in plain words

Every device running our app becomes two things at once: a walkie-talkie **and** a relay tower. If your message cannot reach someone directly, it is passed through other people's devices until it arrives.

Three analogies that explain the whole thing:

Passing a note in class. You cannot reach your friend at the back, so the note goes through three people in between. Nobody in the middle reads it.

Why WhatsApp dies. WhatsApp is a post office. Your letter travels to a sorting centre far away and comes back to the person sitting next to you. If the post office burns down you cannot write to someone in the same room. We skip the post office.

The bucket chain at a fire. After a disaster, connectivity is not zero — it is patchy. One person near a window still has a bar of signal. Right now that helps only them. We build the chain so everyone's messages reach that one person's signal.

The smaller pieces, same idea:

Feature	In plain words
Priority for SOS	An ambulance in traffic — everyone pulls aside, at *every* device it passes through
Smart routing	Picking a road: the shortest one may be full of potholes, so we track which links keep dropping
TTL	An expiry on the note — after ten hands it is torn up, so nothing circles forever
Duplicate check	"I already passed this one," so the room does not flood with copies
Heartbeat	Roll call every three seconds; silent for ten means you have left
Encryption	A sealed envelope — the twenty people who carry it cannot read it
Store-and-forward	No one nearby can take it, so you keep it in your pocket and hand it over when you meet someone who can

The one sentence all three of us should say the same way: after a disaster connectivity isn't zero, it's patchy — one person near a window has a bar, and our system chains devices so that one person's signal becomes everyone's signal.

0.2 Who does what

Member	Semester track	Phase 1 role	Slides
Tanvi Sheth	Track A — Protocol & Networking	Code and live demo	5, 8, 9, 10
Twissha Shah	Track B — Routing & Reliability	Study of existing systems, gap statement	2, 3, 6, 7
Miti Shah	Track C — Platform, Data & Interface	Repo, problem definition, planning pack, deck assembly	1, 4, 11, 12

Tracks are locked for the semester. Each track owns its code **and** its documentation, UML and report sections. Full definitions in §17.1, Phase 1 detail in §24.7.

0.3 Phase 1 order of work

Order matters even without dates. Step 1 must finish before step 3. Steps 3, 4 and 5 run in parallel — nobody waits. Everything from step 9 needs all three pieces done.

#	Who	What
1	Miti	Create repo, add all three, `main` + `develop`, folder structure from §14, `README`, `CONTRIBUTING.md`, `.gitignore`
2	Miti	Push this doc to `docs/project-plan.md`, start `docs/contribution-log.md`
3	Tanvi	`packet.py` — build and read the 52-byte header, plus tests
4	Twissha	Comparison table of existing systems
5	Miti	Problem definition write-up, 1–2 pages
6	Tanvi	`transport.py` + `transport_udp.py` — two nodes talking across two terminals
7	Twissha	Gap statement
8	Miti	Gantt, risk register, execution plan
9	All three	Write your own four slides
10	Miti	Fix the old one-page abstract, department hardware/software sheet
11	Miti	Merge all twelve slides into one deck, one style
12	Tanvi	Dry-run the demo — tests green, hex dump, two terminals
13	All three	One full rehearsal, each presenting their own slides
14	Miti	Tag `v0.1-phase1`, check everyone has commits
15	All three	Present

0.4 Which sections you need

If you are	Read
Tanvi (Track A)	§7 protocol spec, §14 repo layout, §6 architecture, §10 security
Twissha (Track B)	§24.1 survey targets, §8 routing, §9 reliability, §16 metrics
Miti (Track C)	§1–3 framing, §17 schedule, §22 risks, §14 repo, §11–12 data and UI
Anyone, before a viva or presentation	§19 viva answers, §1 the honest statement

0.5 The two rules

Repo before code. Otherwise the whole weekend lands as one commit from one account.

Everyone pushes their own work, documents included. Phase 3 grades commit conventions and Phase 5 grades work distribution — one person pushing everything costs us marks twice.

1. The framing (locked)

Setting: infrastructure failure — earthquake, fire, building collapse, flood, cyclone. Cellular towers and internet links are unavailable, degraded, or overwhelmed.

Core insight the project is built on: after a disaster, connectivity is never uniformly zero. It survives in scattered pockets — someone near a window with one bar, someone with a satellite messenger, one access point still running off a UPS, a rescue team arriving with their own equipment. Normally that connectivity helps only the person holding it.

What this system does: it chains devices together so that one person's connectivity becomes everyone's connectivity, and so that a message from someone with no signal at all can travel through strangers' devices until it reaches whoever can get it out.

The honest statement — repeat this verbatim in the report, the presentation, and the viva:
The mesh currently runs over LAN sockets, which stand in for Bluetooth Low Energy or Wi-Fi Direct. On LAN we restrict each node's visible neighbour set to emulate limited radio range. This lets us develop and measure the routing, prioritisation and reliability logic without radio hardware. Discovery, routing, reliability and encryption are all transport-agnostic — replacing LAN sockets with a radio transport does not change any layer above it.

Do not hide this. Stating a limitation as a design decision is what separates an engineer from a student who did not think it through.

What a "node" is, this semester: any device running the mesh daemon. We deploy on laptops. The wire protocol is platform-independent, so a mobile client is an implementation task, not a redesign.

2. Problem statement

When communication infrastructure fails, existing tools fail with it. WhatsApp, Signal, Telegram and SMS all require a message to reach a server or a tower somewhere else. A working Wi-Fi router with a cut uplink is enough to break every one of them, even though the devices connected to it can still reach each other at full speed.

People trapped in a collapsed or burning building therefore cannot:
signal their position and condition to anyone outside,
share the connectivity of the one person who still has a weak signal,
coordinate with each other about blocked exits, injuries, or trapped survivors,
be located efficiently by rescue teams, who must search room by room.

3. Objective

Design and implement a decentralised communication system in which every device is simultaneously an endpoint and a router, so that messages travel hop-by-hop across neighbouring devices to reach a destination or an exit point, without internet access or any central server — and evaluate it quantitatively under controlled failure conditions.

4. How a message gets out

There are three exit routes. All three are implemented.

Exit 1 — Gateway node. Some node in the mesh still has a link to the outside (weak cellular signal, satellite messenger, a surviving uplink). It advertises itself as a `GATEWAY` in its heartbeat. Routing treats reaching any gateway as reaching the destination. Fifty trapped people share one person's single bar.
Exit 2 — Rescue node walks in. Rescue personnel carry a device running the daemon as a `RESCUE` node. The moment they come within range of any node in the mesh, the entire mesh becomes reachable to them and every pending SOS flows in at once, with sender IDs and last-known locations. They stop searching room by room; the building tells them where people are.
Exit 3 — Store-and-forward (human data carrier). A node with no known path holds undelivered messages instead of dropping them. When someone walks out of the building and their device meets a node with a path, the queue flushes. The person becomes a physical carrier of forty strangers' messages without doing anything or knowing it.
Exit 3 is delay-tolerant networking. It is the least common feature in comparable student projects and the most striking one to demonstrate.
Secondary value: intra-mesh coordination — "east stairwell blocked", "four of us in room 302, one unconscious", "who has water". Real, but not the headline. Do not lead with it.

5. Scope

5.1 In scope (must be complete and working)

Multicast peer discovery with a live neighbour table
Custom binary wire protocol over UDP + TCP
Link-state gossip and distributed graph construction
Dijkstra routing over a weighted cost function (not hop-count)
Priority queue with SOS pre-emption re-applied at every hop
TTL, duplicate detection, ACK with retry and route recomputation, heartbeat failure detection
Node roles: `NORMAL`, `GATEWAY`, `RESCUE`
Store-and-forward queue for unroutable messages
AES-256-GCM payload encryption; Ed25519 signed headers
Per-node SQLite message and event log
Single-process concurrency via `asyncio` (no thread locks)
Network simulator: N virtual nodes, injectable loss, latency and scripted failures
Metrics harness producing delivery ratio, hop distribution, latency by priority, recovery time
React node UI and rescue dashboard, served by each node

5.2 Out of scope (state this openly)

Real Bluetooth LE / Wi-Fi Direct radio transport
NAT traversal or internet fallback
Voice, video, or file transfer
Scale beyond ~50 nodes
Any machine learning — there is no ML problem here, and adding one invites a question with no good answer
Native mobile application (see stretch goals)
Naming your limits is a strength. Vagueness about them is what gets picked apart in a viva.

6. Architecture

6.1 The decision everything else depends on

The mesh engine must never know how bytes physically move.

Define one interface:

```python
class Transport(Protocol):
  async def send(self, node_id: str, data: bytes) -> None: ...
  def on_receive(self, callback: Callable[[str, bytes], None]) -> None: ...
  async def broadcast(self, data: bytes) -> None: ...
  async def start(self) -> None: ...
  async def stop(self) -> None: ...
```

Write two implementations:

`UdpTcpTransport` — real sockets, used for laptop demos

`SimulatedTransport` — pure in-memory, configurable loss, latency, and link matrix

This single boundary buys you three things:

The simulator. 50 nodes in one process, deterministic, fast, no network. This is where your numbers come from.

Real unit tests. Routing and reliability tested without touching sockets.

The honest answer to the LAN objection. Bluetooth support is a third implementation, not a rewrite.

Get this right and the project is buildable. Get it wrong and you will spend three weeks debugging routing bugs through live sockets.

6.2 Layers inside one node

Layer	Responsibility
Discovery	Find neighbours, maintain neighbour table, detect death
Transport	Move bytes between adjacent nodes
Routing	Maintain the graph, compute paths, choose next hop
Reliability	TTL, dedup, ACK, retry, store-and-forward
Crypto	Encrypt/decrypt payloads, sign/verify headers
Storage	Persist every message and event
API	Serve UI, push live state over WebSocket

6.3 Process model

One node = one Python process. Inside it, `asyncio` runs six coroutines concurrently on a single thread:

Discovery task — broadcast `HELLO` every 3 s; listen; update neighbour table; mark nodes dead after 10 s of silence.

Link-state task — every 5 s, send own neighbour list to neighbours; merge incoming reports; trigger Dijkstra recompute on topology change.
Receiver task — accept TCP connections; parse packets; dedup check; decrypt if destination, forward if not.
Sender task — pop from priority queue; resolve next hop; send; track pending ACKs; retry with fresh route on timeout.
Store-and-forward task — every 10 s, retry anything queued with no known path.
API server — FastAPI, serves the UI and holds WebSocket connections open.
Shared state is exactly three structures: the neighbour table, the graph, and the pending-ACK map. Because everything runs on one event loop thread, **no locks are needed and no race conditions are possible.** That is the clean Operating Systems answer, and it is why `asyncio` was chosen over threads.

7. Wire protocol specification

Fixed binary header, variable path and payload, optional signature. Network byte order (big-endian).

7.1 Header (52 bytes)

Offset	Size	Field	Notes
0	2	`magic`	`0x4D45` ("ME") — reject anything else
2	1	`version`	Currently `1`
3	1	`type`	0 `HELLO`, 1 `LINK_STATE`, 2 `DATA`, 3 `ACK`
4	1	`priority`	0 `SOS`, 1 `NORMAL`, 2 `STATUS`
5	1	`ttl`	Decremented at every hop; dropped at 0
6	1	`flags`	bit0 encrypted, bit1 needs_ack, bit2 store_forward_ok, bit3 signed
7	1	`hop_count`	Incremented at every hop
8	16	`msg_id`	UUIDv4 — dedup key and ACK correlation
24	8	`src`	Node ID, ASCII, null-padded
32	8	`dst`	Node ID, or `RESCUE__` / `BCAST___`
40	8	`timestamp`	Unix milliseconds, uint64
48	2	`payload_len`	Bytes
50	1	`path_len`	Number of node IDs in the path list
51	1	`reserved`	Zero

7.2 Body

```
[ path      : path_len × 8 bytes ]   node IDs already traversed
[ payload   : payload_len bytes  ]   AES-256-GCM ciphertext (12-byte nonce prefixed)
[ signature : 64 bytes           ]   Ed25519 over header + path, present if flags bit3
```

The `path` field is deliberately carried on the wire. It gives you loop detection independent of TTL, and it lets the UI show "relayed via 3 nodes" — the moment where a user actually *sees* the mesh working.

7.3 Constants

Constant	Value	Rationale
`HELLO_INTERVAL`	3 s	Fast enough to notice failure, cheap enough to ignore
`NEIGHBOUR_TIMEOUT`	10 s	Roughly 3 missed hellos
`LINK_STATE_INTERVAL`	5 s	Topology convergence within ~15 s on a 5-hop mesh
`DEFAULT_TTL`	10	Comfortably above the diameter of a 50-node mesh
`ACK_TIMEOUT`	4 s	Above worst-case 10-hop latency
`MAX_RETRIES`	3	Then mark failed and move to store-and-forward
`DEDUP_WINDOW`	300 s	Long enough to outlive any retry sequence
`SF_RETRY_INTERVAL`	10 s	Store-and-forward flush attempt

7.4 Transport choice per message type

Type	Transport	Why
`HELLO`	UDP multicast	Inherently "shout to everyone"; loss is irrelevant at 3 s intervals
`LINK_STATE`	UDP unicast to neighbours	Periodic and self-correcting; loss tolerable
`DATA`	TCP per hop	Each hop should be ordered and reliable even though the end-to-end path is not
`ACK`	TCP per hop	Same

Note the reasoning: TCP gives you per-link reliability for free. Implementing that over UDP means reimplementing TCP, badly.

8. Routing

8.1 Building the graph

Link-state, the same family as OSPF. Each node periodically tells its neighbours which neighbours it has and how good each link is. These reports propagate. Within roughly 15 seconds every node holds an approximate graph of the whole mesh: vertices are nodes, edges are active links.

8.2 Cost function

Edge weight is **not** 1. Fewest-hop paths are frequently the flakiest.

```
cost(link) = 1                    # base hop penalty
           + (1 - link_quality) * 5    # weak links are expensive
           + recent_failures * 3       # links that dropped packets are punished
```

`link_quality` = fraction of the last 20 expected `HELLO`s that actually arrived, in [0, 1]. `recent_failures` = ACK timeouts attributed to this link in the last 60 s.

Why this matters and how to prove it: two paths may both be 3 hops, but one traverses a node dropping 40 % of packets. Hop-count routing picks arbitrarily; this picks correctly. Measure both in the simulator and report the delivery-ratio difference. That single comparison is the strongest empirical claim in the project.

8.3 Path selection

Dijkstra from self to destination over the weighted graph. Recompute on any topology change (neighbour added, neighbour lost, link quality crossing a threshold). Cache the result; invalidate on change.

For `dst = RESCUE__`, treat **any** node advertising role `GATEWAY` or `RESCUE` as a valid terminal, and pick the cheapest one. This is what makes the gateway story work.

If Dijkstra returns no path, the message goes to the store-and-forward queue rather than being dropped.

9. Reliability

Each mechanism exists to prevent one specific failure. Learn these as pairs.

Mechanism	Failure it prevents
TTL — decrement at each hop, drop at 0	A misrouted message circulating forever
Dedup set — `msg_id` → first-seen time, expiring after 300 s	One message multiplying into thousands of copies (broadcast storm)
ACK + retry — destination ACKs along the reverse path; sender retries up to 3 times, **recomputing the route each time**	Sender never learning whether help was reached; retrying down a path that has since died
Heartbeat — silent node removed from graph after 10 s	Routing through a device that died ten minutes ago
Path list — loop detected if own ID already present	Loops that TTL alone would only catch after 10 wasted hops
Store-and-forward — queue when no path exists, flush when one appears	Messages dying at TTL expiry when help was 30 seconds away

9.1 Priority handling

Outgoing messages sit in a **priority queue**, not a FIFO. The queue is re-applied at *every* hop, not only at the source. So an SOS does not merely start first — it overtakes ordinary traffic repeatedly along the entire route. That compounding is why the measured latency difference is large, and why it is worth measuring. Priorities: `0 = SOS`, `1 = NORMAL`, `2 = STATUS`. Ties broken by timestamp (oldest first).

10. Security

Threat model: messages transit strangers' devices. Those strangers must not be able to read a sender's location, medical condition, or identity, and must not be able to forge an SOS from someone else's ID.

Concern	Mechanism
Confidentiality	Payload encrypted with AES-256-GCM. Relays read the header (they need `dst` and `ttl`) but the payload is opaque bytes.
Integrity / tamper detection	GCM authentication tag — a relay modifying the payload is detected at the destination.
Forged SOS / Sybil resistance	Header + path signed with Ed25519. Each node's public key is distributed in `HELLO`.
Key establishment	Phase 1: pre-shared session key (get it working). Phase 2: X25519 key agreement per node pair, so no static secret is ever transmitted.

Use the `cryptography` library. Never implement AES yourself. "We used vetted primitives rather than rolling our own" is the correct engineering answer, not a weakness.

11. Data model (SQLite, WAL mode, one file per node)

```sql
CREATE TABLE messages (
 msg_id      TEXT PRIMARY KEY,
 src         TEXT NOT NULL,
 dst         TEXT NOT NULL,
 priority    INTEGER NOT NULL,
 direction   TEXT NOT NULL,      -- sent | received | forwarded
 status      TEXT NOT NULL,      -- pending | delivered | failed | queued_sf
 body        TEXT,               -- plaintext only if we are src or dst
 created_ms  INTEGER NOT NULL,
 resolved_ms INTEGER,
 hop_count   INTEGER,
 path        TEXT                -- comma-separated node IDs
);

CREATE TABLE events (
 id       INTEGER PRIMARY KEY AUTOINCREMENT,
 ts_ms    INTEGER NOT NULL,
 kind     TEXT NOT NULL,         -- sent | recv | forward | drop_ttl | drop_dup
                                 -- | ack | retry | sf_queue | sf_flush
                                 -- | peer_up | peer_down | route_change
 msg_id   TEXT,
 peer     TEXT,
 detail   TEXT
);

CREATE TABLE peers (
 node_id     TEXT PRIMARY KEY,
 role        TEXT NOT NULL,      -- NORMAL | GATEWAY | RESCUE
 last_seen_ms INTEGER NOT NULL,
 link_quality REAL,
 public_key  BLOB
);

CREATE INDEX idx_events_ts ON events(ts_ms);
CREATE INDEX idx_msg_status ON messages(status);
```

WAL mode lets the API read while the engine writes. Use the `sqlite3` standard library with raw SQL — the schema is four tables and an ORM adds indirection without benefit.

This looks like the boring layer. It is the layer that produces your CV bullet, because every number you report is a query against it.

12. User interface

Deliberately minimal. In an emergency nobody reads a menu. Two views, one React codebase, served by each node.

12.1 Node view (`/node`)

Large SOS button. Long-press to arm (prevents accidental fire). Sends a pre-written distress message with node ID and, later, coordinates.

Reachable list. Who is reachable right now and how many hops away.

Message thread. Each message shows live status: `sending → relayed via 3 nodes → delivered`. This hop count is the moment the user *sees* the mesh work — do not hide it.

Nearby count. So people know they are not alone.

Nothing else. Resist adding features; depth is what is being graded.

12.2 Rescue dashboard (`/rescue`)

Live mesh graph via `d3-force`. Nodes appear as they join, grey out as they die, edges thicken with link quality.

SOS queue, sorted by urgency then age, with sender ID, hop path, and time since sent.

Delivery log and live metric strip: delivery ratio, mean hops, nodes online.

12.3 Making it feel like an app

Ship the React build as a **PWA** — manifest plus service worker. It installs to the home screen, has an icon, runs full-screen, and works offline. On demo day it is opened on a phone browser pointing at a node.

Be clear about what this does not do: a browser cannot open raw UDP sockets, cannot do multicast, and cannot use Wi-Fi Direct. A PWA is therefore a *screen for* a node, not a node. Phones as true nodes require native code — see stretch goals.

13. Technology stack and rationale

Layer	Choice	Why this, and what was rejected
Engine	Python 3.11 + `asyncio`	Four concurrent activities per node with zero locks. Java threads would work but cost a week of synchronisation bugs that earn no marks. Python also makes the simulator an afternoon's work.
Discovery	UDP multicast, `asyncio.DatagramProtocol`	Discovery is inherently a broadcast problem.
Per-hop data	TCP, `asyncio.start_server`	Ordered, reliable single hops without reimplementing TCP over UDP.
Crypto	`cryptography` (AES-256-GCM, X25519, Ed25519)	Vetted primitives.
Storage	SQLite (WAL) + `sqlite3` stdlib	File-based, zero server — which is philosophically consistent with the whole project. Postgres would require the very infrastructure being argued against. SQLAlchemy rejected as unnecessary indirection.
Node API	FastAPI + WebSocket	The UI needs *push* (node down, message delivered, SOS in), not polling. Runs on the same event loop as the engine — no IPC, no second process.
UI	React + Vite + Tailwind	One codebase, two routes. Zero time spent on CSS.
Topology view	`d3-force`	Handles nodes appearing and vanishing gracefully — exactly the demo moment.
Simulator	Plain Python, no dependencies	Second `Transport` implementation.
Tests	`pytest` + `pytest-asyncio`	Routing and reliability tested against the simulated transport.

Explicitly rejected:
scikit-learn / ONNX / MiniLM — there is no ML problem in this project. Bolting on a model to look impressive invites "why is this here?" with no good answer.
React Native (Expo) — a second UI codebase, native build tooling, and device permissions, none of which earn marks for networking depth. The engine cannot run inside it anyway.

14. Repository layout



```
mesh/
  core/
    packet.py        # header pack/unpack, validation
    transport.py     # the interface
    transport_udp.py # real sockets
    transport_sim.py # simulated links, loss, latency
    discovery.py     # hello, neighbour table
    routing.py       # graph, link-state merge, dijkstra, cost function
    reliability.py   # ttl, dedup, ack, retry, store-and-forward
    crypto.py        # aes-gcm, ed25519, x25519
    store.py         # sqlite
    node.py          # wires everything together
  api/
    server.py        # fastapi + websocket
  sim/
    scenarios.py     # named scenarios
    runner.py        # runs N nodes, collects metrics
  ui/                # react + vite + tailwind
  tests/
  docs/
    project-plan.md  # this file
  config/
    node_a.yaml ...
```

14.1 Node configuration

```yaml
node_id: "A"
role: NORMAL          # NORMAL | GATEWAY | RESCUE
listen_port: 9001
api_port: 8001
multicast_group: "239.10.10.1"
multicast_port: 9999
visible_neighbours: ["B", "C"]   # range emulation on LAN — omit for full visibility
```

`visible_neighbours` is the range-emulation knob. A node ignores `HELLO`s from anyone not on its list. This is what forces multi-hop behaviour on a flat LAN, and it is the mechanism referenced in the honest statement in section 1.

14.2 Running

```bash
```

```
# Development — one node per terminal
python -m mesh.node --config config/node_a.yaml

# Demo — same command on each laptop, all on one Wi-Fi

# Simulation
python -m mesh.sim.runner --scenario collapse_40 --loss 0.2 --runs 200
```

15. Simulator design

The most valuable engineering artefact in the project, and the least common in comparable work. Build it early — it is how you debug everything else.

What it does: instantiates N `Node` objects in one process, each wired to a `SimulatedTransport`. A link matrix defines who can hear whom. The transport applies configurable packet loss and latency per link, and a scenario script kills and revives nodes on a timeline.

Scenario definition:

```python
Scenario(
 name="collapse_40",
 nodes=40,
 topology="grid",          # grid | random | chain | building
 loss=0.20,
 latency_ms=(5, 40),
 gateways=["G1"],
 events=[
  (t=10, "kill", "N17"),
  (t=25, "kill", "G1"),     # all SOS should now queue in store-and-forward
  (t=60, "revive", "G1"),   # and flush
 ],
 traffic=[
  (t=5, "sos",    "N33", "RESCUE__"),
  (t=5, "normal", "N12", "N28"),
 ],
)
```

Required scenarios: `baseline_10`, `collapse_40`, `no_gateway`, `high_loss`, `chain_10hop`, `hopcount_vs_weighted` (the routing comparison).

16. Evaluation and target metrics

Every number below is a query against the `events` and `messages` tables, produced by the simulator across repeated runs. These are the claims you defend in the viva and put on your CV.

Metric	How measured	Target
Delivery ratio	delivered ÷ sent, 200 runs, 40 nodes, 20 % loss	⩾ 90 %
Median hop count	`hop_count` of delivered messages	Report distribution
SOS latency advantage	median SOS latency vs median NORMAL latency under load	⩾ 3× faster
Recovery time	node death → first successful reroute	⩽ 10 s
Weighted vs hop-count routing	delivery ratio, both modes, identical scenario	Weighted measurably better
Store-and-forward recovery	messages queued during gateway outage that flush on return	100 %
Convergence time	node join → present in every node's graph	⩽ 15 s
Dedup effectiveness	duplicate packets dropped ÷ total received	Report

Target CV sentence — this is what the whole project exists to let you say truthfully:
Across 200 simulated runs on a 40-node mesh with 20 % packet loss, delivery succeeded 94 % of the time with a median 3-hop path; SOS messages arrived 4× faster than routine traffic, and the mesh recovered from node failure in under 8 seconds.

17. Build plan

Eleven weeks. 15 August to 2 November 2026. Phase dates are fixed and the schedule is built backwards from them.

Phase	Date	Assessed by
Phase I	Mon 17 Aug 2026	Project guide
Phase II	Wed 02 Sep 2026	Project guide
Phase III	Wed 30 Sep 2026	Project guide
Phase IV	Mon 12 Oct 2026	Project guide
ESE	Mon 02 Nov 2026	Industry expert / alumni

Week	Dates	Build milestone	Evaluation deliverable
0	Sat 15 – Sun 16 Aug	Repo created, Git conventions, `CONTRIBUTING.md`, `.gitignore`, README	**Phase I pack**: survey table, slides, Gantt, planning doc
1	Mon 17 – Sun 23 Aug	`packet.py` header pack/unpack + tests; `Transport` interface; `UdpTcpTransport`; two nodes exchange a message	**Phase I due Mon 17.** SRS drafting starts
2	Mon 24 – Sun 30 Aug	Discovery (multicast hello, neighbour table, death detection); single-hop relay A→B→C	SRS complete; six UML diagrams
3	Mon 31 Aug – Wed 02 Sep	`SimulatedTransport` + simulator runner skeleton	**Phase II due Wed 02 Sep.** Algorithm write-ups + complexity
4	Thu 03 – Wed 09 Sep	Link-state gossip and graph construction; crypto module in parallel	—
5	Thu 10 – Wed 16 Sep	Dijkstra + weighted cost function; `hopcount_vs_weighted` scenario runs	Module documentation
6	Thu 17 – Wed 23 Sep	TTL, dedup, ACK with retry; priority queue with per-hop pre-emption	—
7	Thu 24 – Tue 30 Sep	Node roles, gateway routing, store-and-forward; SQLite store wired in	**Phase III due Wed 30 Sep.** Tag `v0.3-phase3`, module walkthrough, Git history
8	Wed 01 – Wed 07 Oct	FastAPI + WebSocket; node UI; rescue dashboard; metrics harness	Functional test suite + requirement traceability table
9	Thu 08 – Mon 12 Oct	Integration across five machines; non-functional test runs	**Phase IV due Mon 12 Oct.** Report, tag `v0.4-phase4`
10	Tue 13 – Mon 19 Oct	200-run metrics sweep; all figures generated	Research paper draft
11	Tue 20 – Mon 26 Oct	PWA packaging; UI polish; bug fixes	Paper and poster finalised; contribution logs
12	Tue 27 Oct – Mon 02 Nov	Three clean end-to-end demo rehearsals; buffer	**ESE Mon 02 Nov**

Two rules that survive the compression:

Do not build the UI before week 8. It is still the most common failure mode. But it can no longer wait until the final fortnight, because Phase IV explicitly grades front-end / back-end integration on 12 October.

Documentation runs in parallel with code from week 1, not after it. Phases I and II are almost entirely written deliverables and they land before any substantial code exists.

Casualty of the compressed timeline: the Kotlin client (section 23, stretch goal 1) is unlikely to fit. Attempt it only if week 9 finishes on schedule with the core complete. Do not trade core completeness for it.

17.1 Team split — three tracks

Work in parallel from week 1, against the `Transport` interface. Each track owns **code, documentation, UML, report sections and testing** for its area — not just code. If one person writes all the documentation, the ethics rubric in Phase V will show it.

Track A — Protocol & Networking

Code	`packet.py`, `transport.py`, `transport_udp.py`, `discovery.py`, `crypto.py`
Owns	Wire protocol design, transport selection, security design
UML	Sequence diagram, deployment diagram
Algorithm write-ups	Packet parse/validate, link-state merge
Report sections	7 (protocol), 10 (security), 13–14 (tools, repo)
Testing	Packet round-trip, discovery, all security non-functional tests
Course claim	CE208 Computer Communications and Networks

Track B — Routing & Reliability

Code	`routing.py`, `reliability.py`, priority queue, dedup, store-and-forward
Owns	The graph, Dijkstra, the cost function, all reliability mechanisms
UML	Activity diagram, state machine
Algorithm write-ups	Dijkstra + cost derivation, priority queue, dedup, store-and-forward flush
Report sections	8 (routing), 9 (reliability), 15 (results)
Testing	Reliability under injected loss, **the weighted-vs-hop-count experiment**
Course claim	CE201 Discrete Structures & Graph Theory, CE207 DAA, CE202 Data Structures

Track B owns the project's central empirical result. That is the strongest single item in the ESE and it belongs to one person.

Track C — Platform, Data & Interface

Code	`transport_sim.py`, `sim/runner.py`, `sim/scenarios.py`, `store.py`, `api/server.py`, `ui/`
Owns	Simulator, metrics harness, database, API, both UIs, Git hygiene
UML	Use case diagram, class diagram
Algorithm write-ups	Scenario execution model, metric derivations
Report sections	11 (database), 12 (UI), 16 (evaluation)
Testing	Test infrastructure, traceability table, scalability and performance runs
Course claim	CE204 Database Management, CE206 Operating Systems

Dependency order — who blocks whom

```
Track A (packet + transport)  →  everyone, week 1
Track C (simulator)           →  Track B, week 3
Track B (routing + reliability) →  Track C (metrics), week 7
```

Track A must ship the packet format and `Transport` interface in week 1 or the other two are idle. Track C must ship the simulator by week 3 or Track B debugs through live sockets, which is the single biggest schedule risk in section 22.

Load balancing — why the documentation is split this way

The three tracks peak at different times, so the documentation load is assigned to whoever is light that week:

Weeks	Track A	Track B	Track C
0	Repo, packet design	Survey table	Git setup, Gantt, slides
1–2	**Heavy** — packet, transport, discovery	Light code → **owns SRS + UML + algorithm write-ups**	Simulator scaffolding
3	Phase II support	**Heavy** — Phase II docs	Simulator complete
4–5	Crypto	**Heavy** — graph, Dijkstra, cost function	Scenarios, store schema
6–7	Light → module docs	**Heavy** — reliability, priority, store-forward	SQLite, Phase III prep
8–9	Light → **report assembly**	Integration support	**Heavy** — API, UI, integration, testing
10–11	**Research paper draft**	**Metrics sweep, all figures**	PWA, polish, poster
12	Rehearsal	Rehearsal	Rehearsal

Presentation rotation
Each phase is presented by a different member; the fifth is presented jointly. Phase V grades distribution of work and an external examiner has no other way to see it.

Phase	Presenter
I	Track C (it is a planning-heavy deck)
II	Track B (design and algorithms)
III	Track A (module walkthrough and Git)
IV	Track C (live integration)
V	All three — one section each

Assigned tracks (locked)

Member	Track	Phase 1 role
Tanvi Sheth	Track A — Protocol & Networking	Code and live demo
Twissha Shah	Track B — Routing & Reliability	Study of existing systems, gap statement
Miti Shah	Track C — Platform, Data & Interface	Repo, problem definition, planning pack, deck assembly

Do not swap mid-semester. Each track owns its code, documentation, UML, report sections and testing — see the three track definitions above.

18. Demo script

Five machines. Configure `visible_neighbours` so that L1 cannot see L4 directly.

Machine	Role
L1	Trapped sender, no gateway reachable
L2, L3	Relay nodes, placed apart
L4	Gateway node, **starts switched off**
L5	Rescue dashboard, projected for the panel

Six beats, roughly eight minutes:
Self-organisation. Start the nodes. They appear on the dashboard with no configuration entered.
Priority. Send a NORMAL and an SOS simultaneously from L1. SOS arrives first. Show both hop paths.
Self-healing. Kill L3 mid-transfer. The graph redraws; the next message routes around it.
No path. Kill the gateway. Send five SOS messages. Show them queued in store-and-forward, undelivered.
The flush. Bring the gateway back. All five deliver at once. *This is the moment the panel remembers.*

Scale and evidence. Switch to the simulator: 40 nodes, 20 % loss, and the metrics table.
Also rehearse a failure you understand. Deliberately show a message that fails and explain exactly why. A demo that only ever succeeds is less convincing than one where you can account for the failure.

19. Viva preparation

Learn these in two sentences each.
"How is this different from WhatsApp?" WhatsApp needs a server in a datacentre; no internet means the app is a brick even when your Wi-Fi is perfect. Here the network *is* the users — ten devices in a collapsed building form a working network with zero infrastructure.
"You need Wi-Fi. So why not just use the Wi-Fi?" A router does two separate jobs: connecting local devices to each other, and forwarding traffic to the internet. The second can fail completely while the first works fine. Our system only needs the first; every existing messaging app needs the second.
"But on one router every device can reach every other directly — so the routing is fake." Correct, and we say so. We restrict each node's visible neighbour set to emulate limited radio range, which lets us measure routing behaviour precisely and repeatably. The transport is isolated behind an interface, so a radio transport is a drop-in replacement.
"Why not use Bluetooth mesh, it already exists?" Bluetooth mesh gives you links. What we built is the routing, prioritisation and reliability logic on top of links — and we can demonstrate it under measured failure conditions, which is the part that matters.
"Nobody has this installed before the earthquake." Correct — consumer download is the wrong adoption model. See section 20.
"Why Dijkstra and not just flooding?" Flooding delivers, but its message count grows with the square of the node count and it collapses the network under load. Dijkstra over a weighted graph delivers along one path and lets us prefer reliable links over short ones — which we measured.
"Why asyncio and not threads?" Four concurrent activities share three mutable structures. On one event loop there are no locks and no race conditions, so the concurrency is provably correct by construction rather than by careful locking.

20. Adoption model

Almost no student project addresses how its system reaches people's hands. Include this; it is a differentiator.
The realistic path is **institutional pre-deployment**, not consumer downloads:
Hospitals, campuses, hostels, factories, malls and stadiums pre-install it on staff and resident devices as part of safety compliance. These are precisely the buildings where mass-casualty events occur.
Rescue and civil defence teams carry it as standard kit, so Exit 2 works even if no civilian has it installed.
It ships as a module inside an app people already have — a government disaster app, a campus app, a building-management app.
Note that this brings hospitals and hostels back into the story, but for the correct reason: not "they share a LAN", but "they are institutions that would actually pre-deploy it".

21. Course mapping

Course	Where it genuinely appears
CE208 Computer Communications and Networks	Custom binary protocol, socket programming, TTL/ACK/heartbeat design, transport selection per message type
CE206 Operating Systems	Concurrent tasks per node, event-loop concurrency, shared-state design, non-blocking I/O
CE202 Data Structures	Priority queue, hash set for dedup, adjacency structure, ring buffer for link-quality history
CE201 Discrete Structures and Graph Theory	Mesh as a weighted graph, connectivity, path analysis, diameter and TTL selection
CE207 Design and Analysis of Algorithms	Dijkstra, cost function design, complexity as node count scales, empirical comparison against hop-count
CE204 Database Management	Per-node event and message log, analytical queries producing all reported metrics

Action item: the submitted form ticks CE201, CE204, CE206, CE207, CE208 but leaves CE202 unticked, despite Data Structures being used heavily. Tick it.

22. Risks and mitigations

Risk	Mitigation
Routing bugs debugged through live sockets consume weeks	Simulator by week 3; all routing/reliability work tested in-process
UI work crowds out the graded core	UI is week 8, no earlier. Non-negotiable.
Multicast blocked on college Wi-Fi	Test early (week 2). Fallback: unicast bootstrap to a seed list, still no central server
Scope creep back toward ML or a native app	Section 5.2 is the contract
Demo fails live	Rehearse three clean runs in week 14; keep a recorded backup run
Uneven parallel work	Track boundaries defined at the `Transport` interface, agreed week 1

23. Stretch goals (only after section 5.1 is complete)

Kotlin Android client speaking the same wire protocol. Discovery, send, receive, forward — no routing intelligence needed. It joins the same mesh as the Python nodes. *Why this beats a React Native rewrite:* it proves the protocol is a real protocol rather than a Python API, it costs about a week for one person instead of a full re-implementation, and it makes the demo "three laptops and a phone in one mesh". The CV line writes itself — *designed a language-independent mesh protocol with reference implementations in Python and Kotlin.*

X25519 per-pair key agreement replacing the pre-shared key.

Multi-factor cost function adding battery level and congestion with tunable weights.

Geotagged SOS so the rescue dashboard maps distress by location.

Wi-Fi Direct transport as a third `Transport` implementation — the true removal of all infrastructure.

24. College phase evaluation mapping

Five graded phases. Two rules matter more than anything else here:

Set up Git properly this weekend, not in October. Phase 3 grades branching strategy and commit conventions as a separate rubric point. A repo with 40 commits all made in the last week, mostly by one person, loses marks in Phase 3 *and* in Phase 5 ethics (distribution of work). Git history is the only evidence of fair distribution you will have.

Documentation is graded before the code exists. Phases 1 and 2 are entirely written deliverables. Most of the content already exists in this document — the work is reformatting it into the required templates, not inventing it.

24.1 Phase 1 — Problem Definition · Study of Existing Systems · Presentation · Project Planning

Rubric point	Source	Note
P1 Problem Definition	Sections 1–3	Lead with the pocket-connectivity insight, not "people can't talk"
P2 Study of existing/related systems	**Gap — must be written**	See list below
P3 Presentation	—	10 slides: problem, gap, insight, three exits, architecture, scope, plan
P4 Project Planning	Sections 17, 17.1, 22	Gantt chart, track split, risk register

The comparative survey (the only real new work in Phase 1). Cover these and build one comparison table with columns: infrastructure required, routing method, priority handling, store-and-forward, encryption, availability.

Consumer mesh apps: Bridgefy, FireChat (discontinued), Briar

Dedicated hardware: goTenna Mesh, Meshtastic (LoRa)

Research/humanitarian: Serval Project, DTN / RFC 4838 bundle protocol

Routing protocols: AODV (RFC 3561), OLSR (RFC 3626), B.A.T.M.A.N., Babel

Platform APIs: Android Nearby Connections, Wi-Fi Aware, Bluetooth Mesh Profile

Where your gap statement lands: existing consumer meshes do flooding or simple hop-count routing with no message prioritisation and little published measurement. Your contribution is a priority-aware, link-quality-weighted mesh with delay-tolerant fallback, evaluated empirically. Say exactly that.

24.2 Phase 2 — SRS · UML · Design of Algorithm · Presentation

Rubric point	What to produce
P1 SRS	IEEE 830 format. Functional requirements map one-to-one onto section 5.1. Non-functional: delivery ratio under loss, convergence time, recovery time, confidentiality — all quantified from section 16 targets. Use the same structure as the SE-lab SRS.
P2 UML	Six diagrams, listed below
P3 Design of Algorithm	Pseudocode + complexity for each, listed below
P4 Presentation	Walk the sequence diagram and the routing algorithm — those are the two that show depth

UML set:

Use case — actors: Trapped User, Relay User (passive), Gateway Node, Rescue Coordinator

Class — `Packet`, `Node`, `Transport`, `NeighbourTable`, `RoutingTable`, `MeshGraph`, `PriorityQueue`, `DedupSet`, `StoreForwardQueue`, `CryptoService`, `MessageStore`

Sequence — SOS relayed A→B→C→Gateway, including the ACK returning along the reverse path

Activity — packet receive path: parse → dedup check → TTL check → destination? decrypt : decrement, append path, re-queue

State machine — message lifecycle: `created → queued → in_flight → awaiting_ack → delivered` with branches to `retrying`, `queued_sf`, `failed`, `dropped_ttl`

Deployment — five node processes on separate machines, each with its own SQLite file and served UI; explicitly no server box in the diagram

The state machine and the deployment diagram are the two that make the examiner realise the design is thought through. The deployment diagram having no central server is the whole project in one picture.

Algorithms to write up with complexity:

Algorithm	Complexity
Dijkstra with the weighted cost function	O((V + E) log V) with a binary heap
Link-state merge on receiving a report	O(deg(v))
Priority dequeue / enqueue	O(log n)
Duplicate detection	O(1) average, hash set
Store-and-forward flush attempt	O(q · (V + E) log V) worst case, q = queue length
Packet parse/validate	O(1) — fixed 52-byte header

Include the cost function derivation and justify each coefficient. That is the part that reads as *design* rather than *implementation*.

24.3 Phase 3 — Module-wise Coding · Git Repo · Branching · Presentation

The repo layout in section 14 is already module-wise; each file under `core/` is one gradeable module.

Git conventions — decide these in week 0 and write them into `CONTRIBUTING.md`:

Branches: `main` (tagged releases only) ← `develop` ← `feature/<track>-<short-name>`. Also `fix/…`, `docs/…`, `test/…`

Commits: Conventional Commits — `feat(routing): add link-quality term to cost function`, `fix(reliability): reset ttl check before dedup`, `test(sim): add high_loss scenario`

Every merge into `develop` goes through a pull request reviewed by one other member. Two of you will be tempted to skip this. Don't — the PR trail is Phase 3 and Phase 5 evidence.

Tag each phase submission: `v0.1-phase1`, `v0.2-phase2`, and so on

`.gitignore`, `README.md`, `CONTRIBUTING.md`, `requirements.txt` present from day one

Commit distribution is graded. Aim for a roughly even commit count across the three tracks by Phase 5. Check it periodically with `git shortlog -sn`.

24.4 Phase 4 — Integration · Testing · Report · Presentation

Rubric point	What to produce
P1 Integration	React UI ↔ FastAPI WebSocket ↔ mesh engine, running on five machines
P2 Functional testing	`pytest` suite per module, plus end-to-end scenario tests. Include a traceability table: each functional requirement from the SRS → the test that proves it
P3 Report	Full document; sections 1–23 of this file are the raw material
P4 Presentation	Live integration demo, not slides about integration

Non-functional testing — this is where most teams write nothing and lose the point. Test and report:

Property	Test
Reliability	Delivery ratio at 0 %, 10 %, 20 %, 30 % injected loss
Scalability	10 / 20 / 40 nodes — latency and message overhead growth
Performance	Median and 95th-percentile end-to-end latency by priority
Availability	Recovery time after node death, across 50 trials
Security	Relay cannot decrypt a payload it forwards; a forged-signature SOS is rejected; a tampered payload fails the GCM tag
Usability	Time from screen-on to SOS sent, measured on five people

24.5 Phase 5 — End Semester

Rubric point	How to win it
P1 Problem Definition (originality, novelty, feasibility, societal application)	Novelty = priority-aware weighted routing with DTN fallback, measured. Societal application = section 4 plus the adoption model in section 20. Feasibility = it runs on five laptops in front of them.
P2 Presentation & Ethics (attitude, deadlines, work distribution)	Phase tags in Git, even commit distribution, a one-page contribution log per member. Hit every phase deadline — this is graded directly.
P3 Work Done (survey, design, simulation, experimentation, tools, budget, execution plan)	The simulator is the single strongest item here. "Simulation" and "experimentation" are separate rubric words — the 200-run metrics table covers both. Budget plan below.
P4 Knowledge (Q&A)	Section 19, delivered without notes
P5 Poster / Research paper	See below

Budget plan — do not skip this because it's software. Produce a small table:

Item	Cost
Development hardware	Existing laptops — ₹0 incremental
Software stack	Python, FastAPI, SQLite, React, `cryptography` — all open source, ₹0
Cloud / server infrastructure	₹0 by design — the system has no server
Notional institutional deployment (100-device campus)	Device provisioning + one gateway with satellite uplink — estimate and cite
Development effort	3 members × 14 weeks × ~8 h/week ≈ 336 person-hours

The "₹0 server cost by design" line is worth calling out explicitly — zero infrastructure cost is a genuine consequence of the architecture, not an accident.

Poster or research paper — choose the paper. Suggested framing:
Priority-Aware Link-Quality Routing with Delay-Tolerant Fallback for Infrastructure-Independent Disaster Communication

Structure: introduction and gap (from 24.1) → system design (sections 6–10) → experimental setup (section 15) → results (section 16, including the weighted-vs-hop-count comparison) → limitations (the LAN emulation, stated honestly) → future work (section 23).

The weighted-vs-hop-count comparison is your actual result. Build every figure around it: delivery ratio vs loss rate for both routing modes, latency CDF by priority, recovery time distribution.

24.6 Schedule

The dated week-by-week plan — build milestones and evaluation deliverables in one table, anchored to the five fixed phase dates — is in **section 17**. There is only one schedule; do not maintain a second.

24.7 The 48-hour Phase I plan (15–17 August)

The guide also expects roughly **15–20 % of the work to be shown**, not just documents. The right 15 % is the protocol foundation — not a UI mock-up, which would show the wrong priorities.

What to demonstrate on Monday:

`pytest` running green on the packet header suite

A hex dump of a real 52-byte header with each field labelled

Two node processes in two terminals: A sends, B receives, B prints the parsed header fields and the payload

`git log --graph` showing commits from all three members

That is a working wire protocol and a working transport. It is defensible as the foundation of the system, and everything else in section 5.1 sits on top of it.

Work division — Phase I

Deliverable	Owner	Detail	Est.
Repository and Git setup	**Miti**	Repo created, all three added, `main` + `develop`, `.gitignore`, `requirements.txt`, folder skeleton from §14, `README.md` (framing + track assignment), `CONTRIBUTING.md` (branch naming + Conventional Commits), `docs/project-plan.md` committed, `docs/contribution-log.md` started, tag `v0.1-phase1` at the end	1 h
Problem definition (rubric P1)	**Miti**	1–2 pages from §1–3: setting, why WhatsApp and SMS fail, three exits, objective, honest LAN statement verbatim	1.5 h
Project planning pack (rubric P4)	**Miti**	Gantt against the §17 dates, track split with names, risk register from §22, execution plan one-pager	2 h
Departmental housekeeping	**Miti**	Reconcile the 29 July one-page abstract with the locked framing in §1; hardware/software list in the shared Google Sheet	1 h
Deck assembly and rehearsal	**Miti**	Merge three authors into one voice, enforce one idea per slide, run one full rehearsal	1.5 h
Study of existing systems (rubric P2)	**Twissha**	8–12 systems from §24.1 in a six-column table; verify claims against primary sources rather than summaries	4–5 h
Gap statement	**Twissha**	One paragraph: consumer meshes flood or use plain hop-count, no prioritisation, almost no published measurement — this project adds priority-aware link-quality routing with DTN fallback, measured	0.5 h
The 15–20 % working demo	**Tanvi**	`packet.py` pack/unpack/validate + `tests/test_packet.py` green; labelled hex dump of a real 52-byte header; `transport.py` + `transport_udp.py` with two node processes exchanging a message across two terminals	5–6 h

Sequencing constraint: Miti's repo setup happens first, before Tanvi writes any code. Otherwise the weekend lands in one commit from one account, which is exactly what Phase III and Phase V penalise. Everything else runs in parallel.

Slide ownership — twelve slides, four each

#	Slide	Owner
1	Title, team, project in one line	Miti
2	The problem — infrastructure failure, what breaks	Twissha
3	Why WhatsApp and SMS fail — a router does two separate jobs	Twissha
4	The insight — connectivity survives in pockets	Miti
5	Three exits out of a building	Tanvi
6	Existing systems comparison table	Twissha
7	Gap statement	Twissha
8	Architecture — six layers and the transport boundary	Tanvi
9	Scope: in and out	Tanvi
10	Live demo — packet format, two nodes talking	Tanvi
11	Schedule, Gantt, phase dates	Miti
12	Track split, risks, what Phase II delivers	Miti

Timing, roughly twelve minutes: slides 1–5 at ~45 s each; slides 6–7 at ~90 s each (the survey is the strongest content, give it room); slides 8–9 at ~60 s; slide 10 the live demo at ~3 min, run it rather than screenshot it; slides 11–12 at ~45 s.

Slide 6 will not fit legibly as a twelve-row six-column table. Either cut to eight systems or split it across 6a and 6b, borrowing time from slide 1. Do not shrink the font to fit.

For Phase I each member presents their own slides — it reads as genuine distribution of work. Phases II–IV revert to the single-presenter rotation in §17.1.

Everyone commits their own work from their own account. Do not let one person push everything.

Checklist

- [ ] GitHub repo created, all three members added, `main` and `develop` branches
- [ ] `CONTRIBUTING.md` with branch naming and Conventional Commits
- [ ] `README.md` with the problem statement, the honest statement from §1, and name-to-track assignment
- [ ] `docs/project-plan.md` — this file, committed
- [ ] `docs/contribution-log.md` started
- [ ] Comparative survey table: at least eight systems, six comparison columns
- [ ] Gantt chart against the real dates in §17
- [ ] Risk register (§22) and track split (§17.1) as a one-pager

- [ ] `packet.py` + passing tests
- [ ] `transport.py` + `transport_udp.py`, two-node demo runs
- [ ] Presentation built and rehearsed once
- [ ] Commits from all three accounts
- [ ] Tag `v0.1-phase1`

24.8 Complete deliverable inventory

Every artifact the five phases require, with the status as of 15 August. "Exists" means it is written somewhere in this document and needs reformatting, not authoring.

#	Artifact	Phase	Status
1	Problem definition write-up	I, V	Exists — §1–3
2	Comparative survey of ≥8 existing systems	I, V	**Not written** — targets listed in §24.1
3	Gap statement	I, V	Exists — §24.1
4	Gantt chart against real dates	I, V	**Not drawn** — data in §17
5	Risk register	I	Exists — §22
6	Track / work split	I, V	Exists — §17.1, **names not yet assigned**
7	Phase I presentation (~12 slides)	I	**Not built** — outline in §24.7
8	SRS (IEEE 830)	II	Requirements exist §5.1, §16 — needs formatting
9	Six UML diagrams	II	**Not drawn** — specified in §24.2
10	Six algorithm write-ups with complexity	II	Table exists §24.2 — pseudocode needed
11	Phase II presentation	II	Not built
12	Git repo, branches, conventions, `CONTRIBUTING.md`	III, V	**Not created** — spec in §24.3
13	Module-wise codebase	III	Not written — layout §14
14	Phase III presentation	III	Not built
15	Integrated system on five machines	IV	Not built
16	Functional test suite + traceability table	IV	Not written
17	Non-functional test results (six properties)	IV	Not run — spec §24.4
18	Project report	IV, V	Raw material is §1–23
19	Phase IV presentation	IV	Not built
20	Metrics table, 200-run sweep	V	Not run — targets §16
21	Budget plan	V	Exists — §24.5
22	Research paper	V	Framing exists §24.5
23	Poster (if chosen over paper)	V	Not built

#	Artifact	Phase	Status
24	Per-member contribution log	V	**Not started — begin now**
25	Demo script, rehearsed	V	Exists — §18
26	Viva answers	V	Exists — §19

24.9 Departmental submissions (separate from phase evaluations)

These are administrative and easy to forget:

Software and hardware list in the shared department Google Sheet

One-page abstract, including the components and software list, uploaded to Drive

Filename format: `GroupNo_UIDs` — for this team, `G20_2024300230_2024300231_2024300239`

The one-page abstract must be updated to match the locked framing in §1. The version submitted on 29 July still describes the building/campus setting and the multi-factor routing that has since been scoped down. Reconcile it before Phase I.

24.10 Presentation content per phase

Each phase's presentation is a separate rubric point. Do not reuse the previous deck unchanged.

Phase	The deck must land
I	The gap. Existing systems, what they lack, what you add. End on the plan.
II	The design. Walk the sequence diagram and the routing cost function — those two show depth.
III	The code. Show the module map, then live `git log --graph` and `git shortlog -sn`. Distribution of work is visible here.
IV	The working system. Live integration, then the non-functional numbers. Not slides *about* integration.
V	The story end to end, the six-beat demo (§18), and the measured result. Assessed by an external expert — assume no prior context.

24.11 Report outline (Phase IV)

Abstract

Introduction and motivation — §1–3

Literature and existing systems survey — §24.1

Problem statement and objectives — §2–3

Proposed system — §4–5

System architecture — §6

Protocol design — §7

Routing design and algorithm analysis — §8

Reliability mechanisms — §9

Security design — §10

Database design — §11

User interface — §12
Implementation: tools and technologies — §13–14
Testing: functional and non-functional — §24.4
Results and evaluation — §16
Limitations — the LAN emulation, stated honestly per §1
Conclusion and future scope — §23
References
Appendix: contribution log, budget plan

24.12 Contribution log

One shared file, `docs/contribution-log.md`, updated weekly by each member. Three columns: week, member, what was delivered. Phase V grades distribution of work and it is assessed by an external examiner who has no other way to know who did what. Git history plus this log is your only evidence. Start it in week 0.

25. Definition of done

- [ ] All fourteen items in section 5.1 implemented and tested
- [ ] All six required simulator scenarios run and produce the section 16 metrics table
- [ ] Weighted routing empirically beaten against hop-count routing, with numbers
- [ ] Demo script section 18 rehearsed end to end three times without intervention
- [ ] Every answer in section 19 deliverable in two sentences without notes
- [ ] Report includes the honest statement from section 1 verbatim
- [ ] CE202 ticked on the course mapping form
- [ ] Repository is clean, README explains how to run in under five minutes
- [ ] All five phase submissions delivered on time and tagged in Git
- [ ] Commit distribution roughly even across all three members (`git shortlog -sn`)
- [ ] Comparative survey table of at least eight existing systems
- [ ] Six UML diagrams, six algorithm write-ups with complexity
- [ ] Requirement-to-test traceability table complete
- [ ] Non-functional test results for all six properties in 24.4
- [ ] Budget plan table included
- [ ] Research paper drafted with the weighted-vs-hop-count comparison as the central result